

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр
По дисциплине: «Интеллектуальный анализ данных »
Тема: «РСА»

Выполнил:
Студент 4 курса
Группы ИИ-26
Хомиченко И.А.
Проверил:
Никитин С.К.

Цель: научиться применять метод PCA для осуществления визуализации данных

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Вариант 7

№ варианта	Выборка	Класс
7	hcv+data.zip	Category

Код программы:

```
import zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

DATA_PATH = "hcv+data.zip"
TARGET_COLUMN = "Category"

def prepare_data(path: str = DATA_PATH):
    with zipfile.ZipFile(path, "r") as archive:
        csv_files = [
            name for name in archive.namelist()
            if name.lower().endswith(".csv")
        ]

        if not csv_files:
            raise ValueError("В архиве не найден CSV-файл")

        with archive.open(csv_files[0]) as file:
            df = pd.read_csv(file)

        df.columns = df.columns.str.strip()

        target_col = None
```

```

for col in df.columns:
    if col.lower() == TARGET_COLUMN.lower():
        target_col = col
        break

if target_col is None:
    raise ValueError(
        f"Столбец '{TARGET_COLUMN}' не найден. "
        f"Доступные столбцы: {list(df.columns)}"
    )

unnamed_columns = [
    col for col in df.columns
    if col.lower().startswith("unnamed")
]

df = df.drop(columns=unnamed_columns)

feature_cols = [
    col for col in df.columns
    if col != target_col
]

features = df[feature_cols].copy()

numeric_columns = features.select_dtypes(
    include=np.number
).columns

categorical_columns = features.select_dtypes(
    exclude=np.number
).columns

for col in numeric_columns:
    features[col] = features[col].fillna(
        features[col].mean()
    )

for col in categorical_columns:
    if features[col].isna().any():
        features[col] = features[col].fillna(
            features[col].mode()[0]
        )

features = pd.get_dummies(
    features,
    columns=categorical_columns,
    drop_first=False,
    dtype=float
)

target = df[target_col].astype(str).to_numpy()
data = features.to_numpy(dtype=float)

means = data.mean(axis=0)
stds = data.std(axis=0, ddof=1)
stds[stds == 0] = 1

data = (data - means) / stds

return data, target

```

```

def pca_manual(data: np.ndarray, n_components: int):
    data_centered = data - data.mean(axis=0)

    cov_matrix = np.cov(data_centered, rowvar=False)

    eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

    sort_index = np.argsort(eigenvalues)[::-1]

    eigenvalues_sorted = eigenvalues[sort_index]
    eigenvectors_sorted = eigenvectors[:, sort_index]

    principal_components = eigenvectors_sorted[:, :n_components]

    projected = np.dot(
        data_centered,
        principal_components
    )

    return projected, eigenvalues_sorted

def explained_loss(
    eigenvalues_sorted: np.ndarray,
    n_components: int
) -> float:
    full_info = eigenvalues_sorted.sum()

    reduced_info = eigenvalues_sorted[
        :n_components
    ].sum()

    loss_percent = (
        100 - reduced_info / full_info * 100
    )

    return loss_percent

def pca_sklearn(
    data: np.ndarray,
    n_components: int
):
    pca = PCA(n_components=n_components)

    projected = pca.fit_transform(data)

    return (
        projected,
        pca.explained_variance_ratio_
    )

def get_class_colors(target):
    classes = np.unique(target)

    cmap = plt.get_cmap("tab10")

    return {
        cls: cmap(i % 10)
        for i, cls in enumerate(classes)
    }

```

```

def plot_2d(
    ax,
    projected_2d,
    target,
    title,
    colors
):
    for cls in np.unique(target):
        mask = target == cls

        ax.scatter(
            projected_2d[mask, 0],
            projected_2d[mask, 1],
            color=colors[cls],
            label=str(cls),
            s=30
        )

    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_title(title)
    ax.legend(fontsize=8)

def plot_3d(
    ax,
    projected_3d,
    target,
    title,
    colors
):
    for cls in np.unique(target):
        mask = target == cls

        ax.scatter(
            projected_3d[mask, 0],
            projected_3d[mask, 1],
            projected_3d[mask, 2],
            color=colors[cls],
            label=str(cls),
            s=30
        )

    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_zlabel("PC3")
    ax.set_title(title)
    ax.legend(fontsize=8)

def main():
    data, target = prepare_data()

    colors = get_class_colors(target)

    proj_manual_2d, eigvals = pca_manual(
        data,
        n_components=2
    )

    proj_manual_3d, _ = pca_manual(
        data,
        n_components=3
    )

```

```

loss_manual_2d = explained_loss(
    eigvals,
    2
)

loss_manual_3d = explained_loss(
    eigvals,
    3
)

proj_sklearn_2d, var_ratio_2d = pca_sklearn(
    data,
    n_components=2
)

proj_sklearn_3d, var_ratio_3d = pca_sklearn(
    data,
    n_components=3
)

loss_sklearn_2d = (
    100 - var_ratio_2d.sum() * 100
)

loss_sklearn_3d = (
    100 - var_ratio_3d.sum() * 100
)

print(
    f"Потеря информации "
    f"(ручной PCA, 2 компоненты): "
    f"{loss_manual_2d:.2f}%"
)

print(
    f"Потеря информации "
    f"(ручной PCA, 3 компоненты): "
    f"{loss_manual_3d:.2f}%"
)

print(
    f"Потеря информации "
    f"(sklearn PCA, 2 компоненты): "
    f"{loss_sklearn_2d:.2f}%"
)

print(
    f"Потеря информации "
    f"(sklearn PCA, 3 компоненты): "
    f"{loss_sklearn_3d:.2f}%"
)

fig1, axes1 = plt.subplots(
    1,
    2,
    figsize=(13, 6)
)

plot_2d(
    axes1[0],
    proj_manual_2d,
    target,
    "PCA вручную, 2D",

```

```

        colors
    )

    plot_2d(
        axes1[1],
        proj_sklearn_2d,
        target,
        "PCA sklearn, 2D",
        colors
    )

    fig1.tight_layout()
    fig1.savefig(
        "pca_2d.png",
        dpi=150
    )

    fig2 = plt.figure(
        figsize=(13, 6)
    )

    ax_manual = fig2.add_subplot(
        1,
        2,
        1,
        projection="3d"
    )

    ax_sklearn = fig2.add_subplot(
        1,
        2,
        2,
        projection="3d"
    )

    plot_3d(
        ax_manual,
        proj_manual_3d,
        target,
        "PCA вручную, 3D",
        colors
    )

    plot_3d(
        ax_sklearn,
        proj_sklearn_3d,
        target,
        "PCA sklearn, 3D",
        colors
    )

    fig2.tight_layout()
    fig2.savefig(
        "pca_3d.png",
        dpi=150
    )

    plt.show()

if __name__ == "__main__":
    main()

```

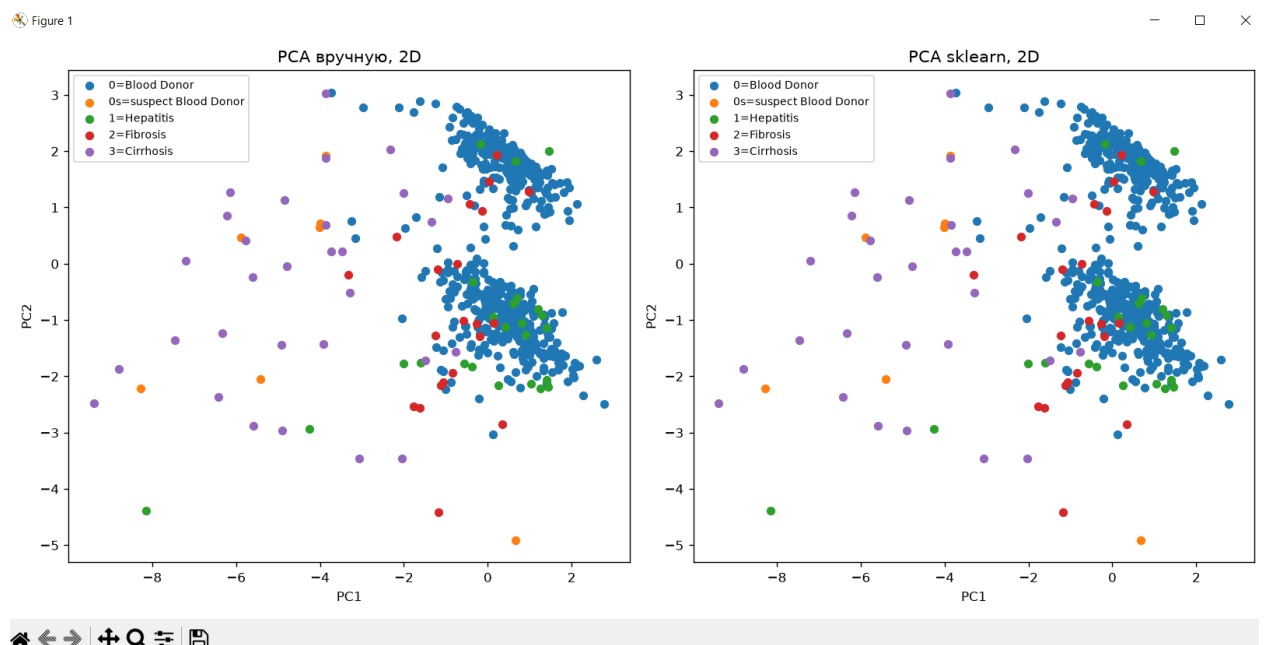
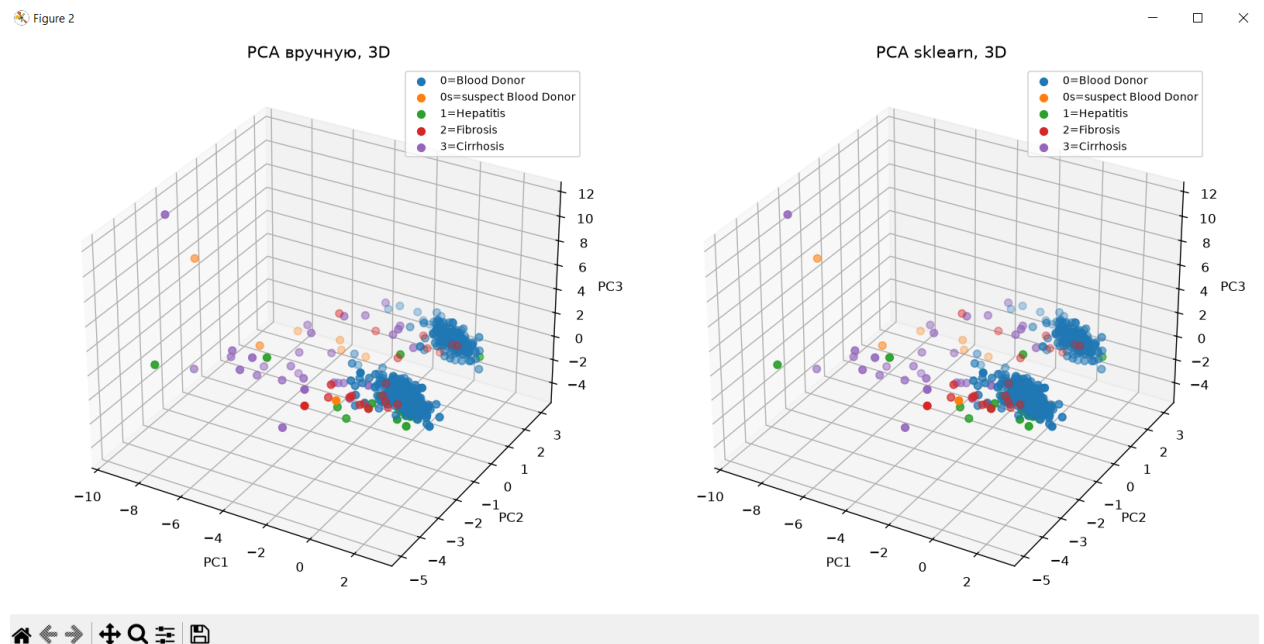
Результат программы:

Потеря информации (ручной PCA, 2 компоненты): 62.86%

Потеря информации (ручной PCA, 3 компоненты): 50.28%

Потеря информации (sklearn PCA, 2 компоненты): 62.86%

Потеря информации (sklearn PCA, 3 компоненты): 50.28%



В результате выполнения работы метод главных компонент был реализован двумя способами — вручную и с помощью `sklearn.decomposition.PCA`. Оба способа дали одинаковые результаты: потери информации составили 62.86% при двух компонентах и 50.28% при трёх, что подтверждает корректность ручной реализации. При этом две компоненты сохраняют около 37.14%

исходной дисперсии, а три — около 49.72%. По графикам видно, что некоторые классы частично группируются, однако между ними сохраняется заметное пересечение. Это показывает, что РСА хорошо подходит для снижения размерности и визуализации данного набора данных, но сам по себе не обеспечивает чёткого разделения всех классов.

Вывод: научился применять метод РСА для осуществления визуализации данных.